

SAGA: A Security Architecture for Governing AI Agentic Systems

Georgios Syros et al. NDSS 2026

Mrunal Shah[2023ucp1838]

Deepesh Ahirwar[2023ucp1692]

Why We Need This

The use of AI Agents is becoming more common. These are programs that can think and act on their own. They work with other programs to get things done. They are easy to use. Can be set up in many different ways like with Anthropic's Agent SDK, OpenAI's Frontier and OpenClaw.. There is a problem. When these AI Agents talk to each other it is not very secure. There are some protocols, like Google's A2A, Microsoft AG2, ANP and AITP that try to make it easier for AI Agents to work together but they do not make sure that all the agents are safe to use.

The Risk of Bad Agents

Where do we find these AI Agents now? There is a risk that a bad agent could attack a good agent when they are working together. Someone could make an agent on purpose or take control of a good agent or even make an agent copy itself many times to cause trouble. If this happens it could hurt the system and make it not work properly.

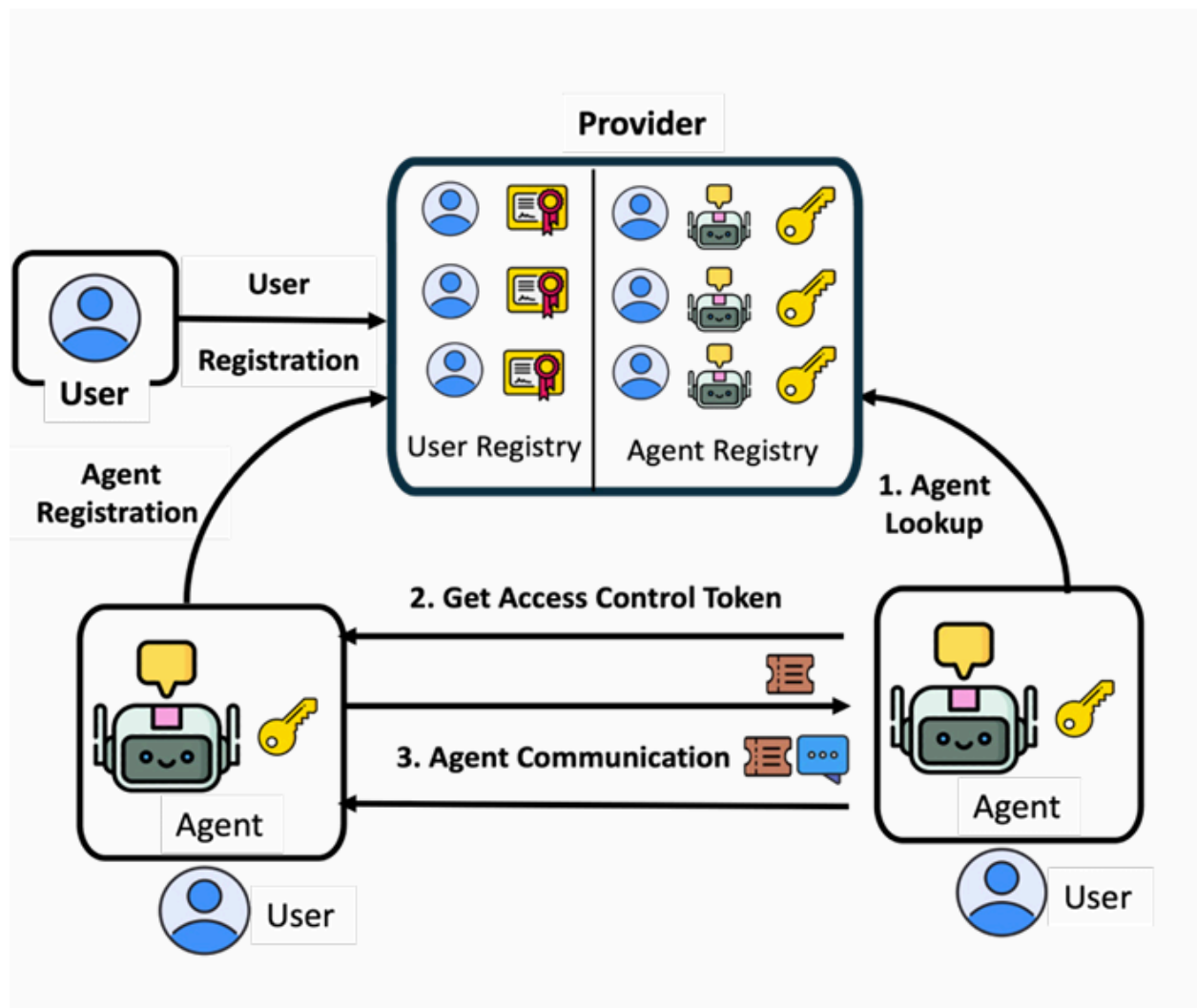
We Need to Control These AI Agents

To make sure these AI Agents are safe we need to make sure they have names and that they can only talk to each other in a secure way. We also need to make sure that people are in charge of what the agents are doing from the time they are made until they are not used anymore. There are some challenges to making this work like how agents find each other how they talk to each other and how we make sure they are only doing what they are supposed to do.

What SAGA Wants to Do

SAGA is a system that wants to help people control their AI Agents and make sure they are only talking to agents that're safe. SAGA wants to make sure that people can decide who their agents can talk to and that bad agents cannot cause any trouble.

How SAGA Works



SAGA has a part that keeps track of all the users and agents and makes sure they are following the rules. Users can register their agents. Decide who they can talk to. Agents can ask to talk to agents and SAGA makes sure it is safe before it lets them.

Registering Users

When someone wants to use SAGA they have to register. The system gives them a name and a way to prove who they are. This makes sure that the user is real and that they can be held responsible for what their agents do.

Registering Agents

When a user wants to register an agent they have to give some information, about the agent, like its name and what it can do. The user also has to say who the agent can talk to and what it can do when it talks to them. SAGA checks all this information. Makes sure it is correct before it lets the agent start working.

Agents Talking to Each Other

When one agent wants to talk to another agent it has to ask SAGA. If SAGA says it is okay the agent gets a key that lets it talk to the other agent. The agents can then talk to each other in a way and SAGA makes sure they are following the rules.

Making Sure SAGA Works Well

SAGA is designed to work even if something goes wrong. It can handle a lot of users and agents. It can recover if something bad happens. It also works with systems like Google's Agent2Agent protocol.

Keeping SAGA Safe

SAGA is a secure system. It uses math to make sure that only the right people and agents can talk to each other. It also makes sure that if someone tries to cheat they will not be able to.

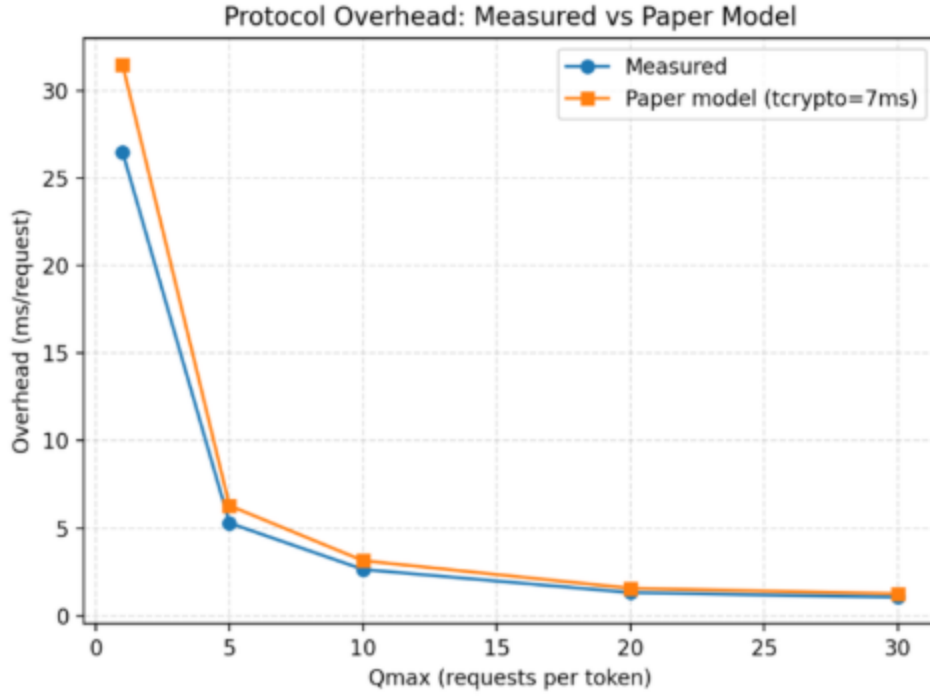
Protecting Against Things

SAGA protects against many different kinds of attacks. It makes sure that only real users can register agents and that agents cannot pretend to be someone. It also makes sure that agents cannot talk to each other if they are not supposed to and that they cannot do things they are not supposed to do. Overall SAGA is a safe system that can protect against many different kinds of threats.

Results Reimplementation

Assumptions:

- Cryptographic processing time assumed as (from paper)
- Analytical overhead computed using the paper's protocol model
- Local deployment used for measured experiments
- Increasing amortizes communication and cryptographic costs across requests



$$c_{\text{proto}}(m) = (\text{RTT}_{B,P} + t_{\text{crypto}}) \cdot \left\lceil \frac{m}{Q_{\text{max}}} \right\rceil, \quad (1) \quad \bar{c}_{\text{proto}}(m) = \frac{c_{\text{proto}}(m)}{m}$$

Metric	Value
Provider RTT median	31.948 ms
Measured crypto cost	2.3466 ms
Amortized overhead (Qmax=1)	34.2945 ms
Amortized overhead (Qmax=5)	6.8589 ms
Amortized overhead (Qmax=10)	3.4295 ms
Amortized overhead (Qmax=20)	1.7147 ms
Amortized overhead (Qmax=30)	1.3718 ms

Evaluation Data

source,qmax,rtt_ms,tcrypto_ms,m_requests,amortized_ms_per_request				
measured,1,24.482,2.0134,100,26.495				
measured,5,24.482,2.0134,100,5.299				
measured,10,24.482,2.0134,100,2.6495				
measured,20,24.482,2.0134,100,1.3248				
measured,30,24.482,2.0134,100,1.0598				
paper_model,1,24.482,7.0,100,31.482				
paper_model,5,24.482,7.0,100,6.2964				
paper_model,10,24.482,7.0,100,3.1482				
paper_model,20,24.482,7.0,100,1.5741				
paper_model,30,24.482,7.0,100,1.2593				

ATTACKS:

A1: If the TLS identity material is missing, the connection should be rejected at or near the handshake.

A2: If there are no access-control credentials (OTK/token), the contact attempt should fail authorization.

A3: If an expired or reused token is used, the initial channel may open, but interaction should be denied.

A4: If there is an impersonation attempt using copied provider-signed public information, marked as TODO in code comments, this behavior is for demonstration only and not fully secure.

A5: If a stolen token is reused (import_token), it should check token binding and ownership protections.

A6: If there is a contact-policy violation, the unauthorized caller should be denied according to the rulebook.

A8: A valid token combined with a valid TLS case shows the limited vulnerability window until the token expires or the quota is depleted. This is the expected behavior of allowing access and then limiting it.

Attack Case	Listener Command	Query Command
A1	<pre>../.venv/bin/python adversary.py listen /private/tmp/bob_attack.yaml</pre>	<pre>../.venv/bin/python adversary.py query /private/tmp/mallory_attack.yaml 1 /private/tmp/bob_attack.yaml 1</pre>
A2	<pre>../.venv/bin/python adversary.py listen /private/tmp/bob_attack.yaml</pre>	<pre>../.venv/bin/python adversary.py query /private/tmp/mallory_attack.yaml 1 /private/tmp/bob_attack.yaml 2</pre>
A3	<pre>../.venv/bin/python adversary.py listen /private/tmp/bob_attack.yaml</pre>	<pre>../.venv/bin/python adversary.py query /private/tmp/mallory_attack.yaml 1 /private/tmp/bob_attack.yaml 3</pre>
A4	<pre>../.venv/bin/python adversary.py listen /private/tmp/bob_attack.yaml</pre>	<pre>../.venv/bin/python adversary.py query /private/tmp/mallory_attack.yaml 1 /private/tmp/bob_attack.yaml 4</pre>
A5	<pre>../.venv/bin/python adversary.py listen /private/tmp/bob_attack.yaml placeholder 5</pre>	<pre>../.venv/bin/python adversary.py query /private/tmp/alice_attack.yaml /private/tmp/bob_attack.yaml 5 /private/tmp/mallory_attack.yaml 1</pre>
A6	<pre>../.venv/bin/python adversary.py listen /private/tmp/candice_attack. yaml</pre>	<pre>../.venv/bin/python adversary.py query /private/tmp/mallory_attack.yaml 1</pre>

A8

```
../.venv/bin/python  
adversary.py listen  
/private/tmp/bob_attack.yaml
```

```
/private/tmp/candice_attack.yaml  
1 6
```

```
../.venv/bin/python  
adversary.py query  
/private/tmp/mallory_attack.yaml  
1 /private/tmp/bob_attack.yaml  
8
```

Evidence Snippets

None

```
[ADVERSARY] Connecting to bob.attack@mail.com:dummy_agent *without*  
TLS credentials.  
[NETWORK] Connected to 127.0.0.1:7201 with verified certificate.  
[AGENT] Sent: 'Hello world!'  
...  
[AGENT] Task deemed complete from receiving side.
```

None

```
Exception: Acces control failed: no one-time key provided from  
initiating agent.
```

None

```
[ADVERSARY] Token is going to be reused.  
[ACCESS] Found token for bob.attack@mail.com:dummy_agent. Will use  
it.  
[ERROR] Error receiving data: Expecting value: line 1 column 1  
(char 0)
```

None

```
[ADVERSARY] Sending wrong stamp: ...
```

```
[NETWORK] Connected to 127.0.0.1:7201 with verified certificate.  
...  
[AGENT] Task deemed complete from receiving side.
```

None

```
FileNotFoundError: [Errno 2] No such file or directory:  
'notmy.token'
```

None

```
TypeError: a bytes-like object is required, not 'NoneType'
```

None

```
[ACCESS] Access denied to candice.attack@mail.com:dummy_agent.  
{'message': 'Access denied.'}  
[ACCESS] Access to candice.attack@mail.com:dummy_agent denied.
```

None

```
[AGENT] Sent: 'MALICIOUS QUERY'  
[ACCESS] Remaining token quota: 48  
[AGENT] Sent: 'MALICIOUS QUERY'  
[ACCESS] Remaining token quota: 47  
...  
[AGENT] Task deemed complete from receiving side.  
[ACCESS] Token invalidated from the receiving side.
```

NOVEL WORK: Merkle-Based Compromise Propagation Tracking

Right now in SAGA to determine exposure after compromise, you'd need to:

- inspect provider logs,
- reconstruct communication graphs,
- correlate timestamps,
- parse interaction metadata.
- Possible log tampering by a compromised provider.
- Poor scalability as agent count grows.
- No cryptographic integrity guarantees for interaction history.

Our Solution - Merkleized Agent Interaction Ledger

Each interaction creates a hash:

$H1 = \text{hash}(A, B, \text{timestamp}, \text{metadata})$

$H2 = \text{hash}(B, C, \text{timestamp}, \text{metadata})$

$H3 = \text{hash}(C, D, \text{timestamp}, \text{metadata})$

What we do

- record every inter-agent interaction,
- maintain cryptographically linked interaction history,
- support efficient compromise tracing,
- support tamper-evident logs,

- allow recursive exposure graph traversal,
- minimize verification overhead.

Hash Chain

Every interaction record references the previous interaction hash.

$$H_n = \text{SHA256}(\text{source_agent} || \text{destination_agent} || \text{timestamp} || \text{session_id} || \text{payload_digest} || \text{previous_hash})$$

This provides:

- chronological integrity,
- append-only semantics,
- tamper detection.

Periodic Merkle Trees

Every fixed-size batch of interactions (example: every 100 interactions) construct a tree over interaction hashes compute a Merkle root, persist the root.

This provides:

- efficient logarithmic verification,
- compact proofs,
- scalable integrity validation.

Interaction Ledger

Create a persistent interaction ledger. Each interaction record should contain:

Persist records in MongoDB (preferred), OR append-only JSON log.

Merkle Tree Builder

periodic Merkle tree generation storage of Merkle roots, Merkle proof generation.

Exposure Graph Reconstruction

If Agent A compromised system should recursively identify:

- all agents contacted by A,
- all downstream agents,
- timestamps of exposure,
- interaction lineage.

Implement:

BFS/DFS traversal over interaction graph, recursive exposure propagation.

Tamper Detection

System must detect:

- modified interaction logs,
- deleted interactions,
- reordered interactions.

Verification should include:

- hash-chain validation,
- Merkle root validation.

Required API:

`verify_ledger_integrity()`

CODE IMPLEMENTATION

Ledger directory: reports/security/demo_merkle/

Ledger file: reports/security/demo_merkle/interaction_ledger.jsonl

Merkle roots file: reports/security/demo_merkle/merkle_roots.jsonl

Simulated Entries (for scheduling event)

```
ledger = InteractionLedger(ledger_dir=ledger_dir, batch_size=4)
entries = [
    ('agent_A', 'agent_B', 'session-001', 'A asks B for calendar availability',
1715342400.0),
    ('agent_B', 'agent_C', 'session-002', 'B delegates email lookup to C',
1715342460.0),
    ('agent_A', 'agent_D', 'session-003', 'A asks D for document context',
1715342520.0),
    ('agent_C', 'agent_E', 'session-004', 'C asks E to summarize attachment',
1715342580.0),
    ('agent_D', 'agent_F', 'session-005', 'D requests final citation check',
1715342640.0),
]
```

Interaction Record

Source	Destination	Session	Timestamp	Interaction Hash
agent_A	agent_B	session-001	1715342400.0	d9624cf887c6d946e7f062986c81ffc2a96feca5193c66af83fd5bc6e03a8ad8
agent_B	agent_C	session-002	1715342460.0	5b62be6bb407783f457b0f0d7c8e3c1e60fd2ea6d42e2ef283c5608d9cac05e8
agent_A	agent_D	session-003	1715342520.0	4a68a9621f92a344837bc0bdf03161af825e7d18a0bb66351372873cd83902d3
agent_C	agent_E	session-004	1715342580.0	f7d5cfa9192897c21e9f7f0bf43c467826fd9b4cb83720cf472315a64f567c18
agent_D	agent_F	session-005	1715342640.0	178cda4c52a7aa2290d573f6a4d7652ce2b36807e8cab05146a65fe0cacdbe74

Printed Merkle Tree

```
Level 0 (5 nodes)
[0] d9624cf887c6d946e7f062986c81ffc2a96feca5193c66af83fd5bc6e03a8ad8
[1] 5b62be6bb407783f457b0f0d7c8e3c1e60fd2ea6d42e2ef283c5608d9cac05e8
[2] 4a68a9621f92a344837bc0bdf03161af825e7d18a0bb66351372873cd83902d3
[3] f7d5cfa9192897c21e9f7f0bf43c467826fd9b4cb83720cf472315a64f567c18
[4] 178cda4c52a7aa2290d573f6a4d7652ce2b36807e8cab05146a65fe0cacdbe74

Level 1 (3 nodes)
[0] acc04b0f97196ffdba24bde1233a64b2a764b93f11e586f31d681c6c9fa37a67
[1] e38aa31bcd46651c78926a9567959ef73b4e288819e21a8767ce525fbf73e758
[2] 1220f2e77c79d7d45e735d6f6fb8cd1aec1a556e04438981f54efa5066cd2ccd

Level 2 (2 nodes)
[0] 1c6d13e25bef4b10cd8bda23d3904d61f782e6112c6bac24cc184ac650343d48
[1] a03b93c44c87b6f51cff441935577aaa19978a8dc86bb6de370b1112121516f5

Level 3 (1 nodes)
[0] 7a577cb82d865376996c80a03d5a48d4cabb9109583a8aa2d3b6c8773bcc46aa
```

Merkle proof

ROOT: 7a577cb82d865376996c80a03d5a48d4cabb9109583a8aa2d3b6c8773bcc46aa

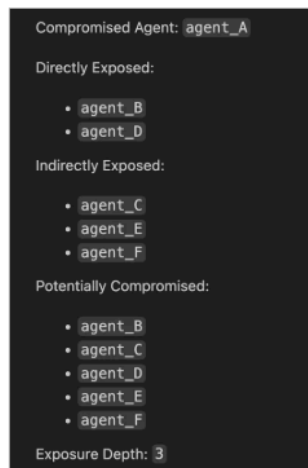
TARGET: 4a68a9621f92a344837bc0bdf03161af825e7d18a0bb66351372873cd83902d3

PROF ROOT: 1c6d13e25bef4b10cd8bda23d3904d61f782e6112c6bac24cc184ac650343d48

Integrity Verification

```
[
  {
    "position": "right",
    "hash": "f7d5cfa9192897c21e9f7f0bf43c467826fd9b4cb83720cf472315a64f567c18"
  },
  {
    "position": "left",
    "hash": "acc04b0f97196ffdba24bde1233a64b2a764b93f11e586f31d681c6c9fa37a67"
  }
]
```

```
{
  "valid": true,
  "record_count": 5,
  "latest_hash": "178cda4c52a7aa2290d573f6a4d7652ce2b36807e8cab05146a65fe0cacdbe74",
  "merkle_root": "7a577cb82d865376996c80a03d5a48d4cabb9109583a8aa2d3b6c8773bcc46aa",
  "errors": []
}
```



`def build_exposure_graph()`

```
agent_A
├── agent_B
│   ├── agent_C
│   │   └── agent_E
└── agent_D
    └── agent_F
```